AIR UNIVERSITY, ISLAMABAD

National Cyber Security Academy

Operating Systems (CY-325)

Assignment No.1

Project Proposal & Deliverables

ZeroTrustOS — Agent-Governed Process Access Control System

Student Name	:	Mahaavia Imran
Registration ID	:	241500
Class & Section	:	BSCYS-F24-IV-B
Course Instructor	:	Sir Balawal Shabir
Department	:	Cyber Security
Date of Submission	:	March 1, 2026

1 Problem Definition

Traditional Operating System security relies on static Discretionary Access Control (DAC) and Mandatory Access Control (MAC) models. The core assumption is implicit trust: if an entity is inside the system perimeter, it is trusted. This paradigm is highly vulnerable to privilege escalation, lateral movement, and fileless malware. **ZeroTrustOS** reverses this philosophy to "Never Trust, Always Verify, Continuously Monitor." Every process, user session, and resource access must be verified in real-time by a system of intelligent agents before execution.

2 Literature Review

Recent cybersecurity research emphasizes continuous authentication and behavioral monitoring aligned with Zero Trust principles.

- **NIST SP 800-207:** Defines Zero Trust Architecture and continuous verification models.
- **NIST AI Risk Management Framework (AI RMF 1.0):** Governs trustworthy AI-based anomaly detection systems.
- **OWASP Top 10 for LLM Applications:** Highlights risks such as prompt injection and excessive AI agency.
- **Contemporary Research (2025–2026):** Focuses on agent-based orchestration and AI-driven anomaly detection for insider threat mitigation.

These frameworks directly influence the design of ZeroTrustOS by ensuring secure orchestration, controlled AI behavior, and least-privilege enforcement.

3 System Architecture & Workflow

The ZeroTrustOS framework operates on a continuous verification loop driven by four specialized agents and a central orchestrator.

- **Identity Agent:** Intercepts login attempts (via Linux PAM) and assigns dynamic trust scores to processes at spawn time.
- **Policy Agent:** Maintains a YAML-based policy store and translates least-privilege decisions into AppArmor/SELinux profiles dynamically.
- **Anomaly Agent:** Establishes behavioral baselines to detect real-time deviations (e.g., unexpected network sockets or memory access).

- **Audit Agent:** Maintains a tamper-proof, append-only SQLite log with SHA-256 hash chains for strict accountability.
- **Orchestrator (Master Agent):** The central decision engine that evaluates verdicts from the Identity, Policy, and Anomaly agents to either ALLOW, DENY, or TERMINATE a request.

4 Agent Interaction Model

The following 10 scenarios outline the interaction across the multi-agent system:

1. **Login Auth:** User logs in → Identity verifies → Policy checks hours → Orchestrator decides.
2. **Process Spawn:** Process spawns child → Identity dynamically re-evaluates the child's trust score.
3. **Unauthorized Read:** Web server attempts to read `/etc/passwd` → Policy blocks → Audit logs.
4. **Privilege Escalation:** Anomaly detects UID change (e.g., 1000 to 0) → Orchestrator terminates process.
5. **Admin Sudo:** Admin runs `sudo su` → Identity re-verifies → Policy enforces rules → Audit records.
6. **Cron Abuse:** New cron job added by `www-data` → Anomaly flags → Orchestrator denies and alerts.
7. **Brute Force:** SSH brute force detected → Identity increments fail counter → Lockout triggered.
8. **Lateral Movement:** Process opens unexpected outbound socket → Anomaly flags → Process isolated.
9. **Tampering Attempt:** Log tamper detected by Audit (hash chain break) → Critical alert to Orchestrator.
10. **Policy Update:** Admin pushes update → Policy dynamically reloads rules → Audit logs change.

5 Agent Interaction Matrix

Scenario / Action	Trigger Event	ID	Policy	Anomaly	Audit
1. Login Authentication	PAM Authentication Hook				
2. Process Spawns Child	`execve()` / `clone()` syscall				
3. Unauthorized File Read	AppArmor Profile Violation				
4. Privilege Escalation	Unexpected UID transition				
5. Admin `sudo` Execution	`sudo` command invocation				
6. Unauthorized Cron Job	Inotify on `/var/spool/cron`				
7. SSH Brute Force	Repeated PAM auth failures				
8. Lateral Movement	Unexpected socket creation				
9. Log Tamper Attempt	Attempted SQLite `UPDATE`				
10. Dynamic Policy Update	YAML configuration modification				

6 System Control Mapping (Governance)

Governance is strictly managed by the **Orchestrator (Master Agent)** using an "All-Pass" consensus model.

- **Consensus (Allow):** All agents return positive verdicts; Orchestrator permits execution and signals Audit.
- **Partial Failure (Deny):** Any agent flags the request; Orchestrator blocks execution and signals Audit.
- **Critical Failure (Terminate):** Anomaly detects severe violation; Orchestrator immediately SIGKILLs the process.

7 Framework Mappings

Standard / Framework	ZeroTrustOS Implementation Clause
NIST ZT (SP 800-207)	*Continuous Verification:* Identity Agent. *Least Privilege:* Policy Agent. *Micro-segmentation:* AppArmor.
NIST AI RMF	*Measure / Manage:* Anomaly Agent's behavioral baselines mapping system metrics.
OWASP LLM Top 10	*LLM01 (Prompt Injection):* Orchestrator sanitizes inputs from agents to prevent malicious syscall injections.

8 Implementation Stack

- **Python:** Chosen for rapid agent development and rich OS-level interaction (`psutil`, `subprocess`).
- **Linux PAM & AppArmor:** PAM intercepts root-level authentication; AppArmor provides readable MAC profiles.
- **SQLite:** Lightweight database enforcing append-only logic for tamper-proof audit hash chains.
- **asyncio / Redis:** Ensures non-blocking, asynchronous communication between the Master Orchestrator and agents.